

CMPU 3010 Databases 2

Lab 1: CommunityCare Database

PostgreSQL setup, database investigation and SQL review

1. Lab Objectives

Estimated time: 60–70 minutes

By the end of this lab you should be able to:

- set up or reuse a PostgreSQL environment;
- create and populate a schema we will use throughout the module (communitycare) from supplied SQL scripts;
- inspect an unfamiliar relational database and infer its structure;
- use SQL skills from Databases 1 to answer operational questions;
- identify database problems that will motivate topics later in Databases 2.

2. Files supplied

Download the files supplied with this lab. You should have:

```
Lab01/  
├── docker-compose.yml  
├── 01-communitycare-schema.sql  
└── 02-communitycare-data.sql
```

The SQL scripts are:

- **01-communitycare-schema.sql** — creates the **communitycare** schema, tables and constraints.
- **02-communitycare-data.sql** — inserts the synthetic sample data used in this lab.

Note: The `docker-compose.yml` file is only required if you choose the Docker setup option.

3. Connect to PostgreSQL

For this lab you need a connection to a PostgreSQL server. Choose **ONE** of the following options. Once you have successfully connected to PostgreSQL, **everyone should continue to Section 4.**

Option A — Reuse PostgreSQL from Databases 1

If you already have a working PostgreSQL installation from Databases 1, you may reuse it.

1. Start PostgreSQL if necessary.
2. Connect using pgAdmin.
3. Connect to the postgres database.
4. Test that your connection works.
5. Continue to **Section 4 – Setup CommunityCare.**

You do not need to create a new database.

Option B — Docker

Use this option if Docker Desktop is installed and running.

Step 1 — Start PostgreSQL

- Place docker-compose.yml in the folder you want to use for this module.
- Open a terminal in that folder and run: `docker compose up -d`
- Docker will start PostgreSQL and pgAdmin.
- It does **not** create the communitycare tables or load the data. You will do this yourself in Section 4.

Step 2 — Connect to PostgreSQL

You can use either pgAdmin or DBeaver (You can download DBeaver Community edition here: <https://dbeaver.io/download/>) or another DB Client if you prefer.

- **Using pgAdmin**
 - Open pgAdmin in your browser at: `localhost:8080`
 - Log in using the details supplied in `docker-compose.yml`.
 - Choose: **Servers → Register → Server**
 - On the **General** tab, enter a name such as CommunityCare
 - On the **Connection** tab enter:

Setting	Value
Host	postgres
Port	5432
Maintenance database	postgres
Username	postgres
Password	postgres

- **Using DBeaver/other DB Client**
 - Create a new PostgreSQL connection using:

Setting	Value
Host	localhost
Port	5432
Database	postgres
Username	postgres
Password	postgres

Step 3 - Test the connection.

- Once connected, continue to **Section 4 – Set up CommunityCare**.

Option C — Supabase

Supabase provides a hosted PostgreSQL database. Use this option if you do not want to install PostgreSQL or Docker locally.

Step 1 — Create a Supabase account

- Go to [Supabase](https://supabase.com/) - <https://supabase.com/> and create a free account.
- Sign in to the Supabase Dashboard.

Step 2 — Create a project

- Select **New Project** and create a project for this module.
- Use a name such as:
- CMPT3010-Databases2
- When prompted, create a **database password** and keep a copy of it.
- Choose a European region where available and create the project.
- It may take a short time for Supabase to provision your PostgreSQL database.

Step 3 — Connect to PostgreSQL

- The simplest option is to use the **SQL Editor** provided in the Supabase Dashboard.
- Open from left-hand floating menu: **SQL Editor** → **New query**
- You are now ready to continue to **Section 4 – Set up CommunityCare**.

4. Setup CommunityCare

Once you have a working connection to PostgreSQL, complete the following steps. These instructions are the same regardless of whether you are using **local PostgreSQL**, **Docker** or **Supabase**.

Step 1 — Open a SQL Editor

- Open a new SQL query/editor connected to the postgres database.
 - **pgAdmin**: select the postgres database and open **Query Tool**.
 - **Supabase**: open **SQL Editor** → **New query**.

Step 2 — Run the CommunityCare Schema Script

- Open the supplied file: `01-communitycare-schema.sql`
- Copy/open the script in your SQL editor and execute the **entire script**.
- The script will automatically:
 - create a schema called communitycare;
 - set communitycare as the working schema;
 - create the seven CommunityCare tables;
 - create the primary keys;
 - create the foreign keys;
 - create the other constraints required by the database.
- You do **not** need to create a database or schema manually.

Step 3 — Check That the Tables Were Created

- Run:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'communitycare'
      AND table_type = 'BASE TABLE'
ORDER BY table_name;
```
- You should see seven tables:
appointment
centre
patient
payment
service
staff
staff_service
- If you do not see these tables, check that 01-communitycare-schema.sql completed without errors before continuing.

Step 4 — Load the CommunityCare Data

- Open the second supplied file: 02-communitycare-data.sql
- Execute the **entire script**.
- This inserts the synthetic CommunityCare data that will be used in the lab.

Step 5 — Set CommunityCare as Your Working Schema

At the beginning of your SQL work from now on, run:

```
SET search_path TO communitycare;
```

This tells PostgreSQL to look in the communitycare schema by default.

For example, instead of writing:

```
SELECT *
FROM communitycare.patient;
```

you can now simply write:

```
SELECT *
FROM patient;
```

Note: SET search_path applies to your current database session. If you reconnect later, run it again before starting your queries.

Step 6 — Check your Setup

- Run:

```
SET search_path TO communitycare;
```

```
SELECT COUNT(*) AS number_of_patients  
FROM patient;
```

```
SELECT COUNT(*) AS number_of_appointments  
FROM appointment;
```

If both queries execute successfully and return counts greater than zero, you are ready to begin **5. Part A — Investigate the Database**.

5. Part A — Investigate the database

Do not examine 01-communitycare-schema.sql yet.

Imagine that you have been given access to an existing database without being given its design documentation. Use a DBClient (if using Docker), pgAdmin, Supabase or PostgreSQL's metadata to investigate it.

Using PostgreSQL metadata

- PostgreSQL stores information about tables, columns, keys and constraints in the `information_schema`. You can query this just like other database tables.
- For example, to investigate the columns of a table:

```
SELECT *  
FROM information_schema.columns  
WHERE table_schema = 'communitycare'  
      AND table_name = 'patient';
```

- Useful Views include:

View	Useful for
<code>information_schema.columns</code>	Columns, data types and NULL settings
<code>information_schema.table_constraints</code>	PK, FK, UNIQUE and CHECK constraints
<code>information_schema.key_column_usage</code>	Columns involved in PK, FK and UNIQUE constraints
<code>information_schema.constraint_column_usage</code>	Columns referenced/used by constraints

Using PGAdmin

- Under the schema you can navigate to Tables.
- Click/double-click/right-click on a tablename to explore the details of the table.

Using Supabase

- Select Database from the left-hand floating menu
- Select Tables from the available options.

Complete the following tasks

Task A1: Identify the **primary key** of each of the seven tables.

Task A2: Identify the **foreign-key relationships** between the tables.

Task A3: Examine the table **staff_service**.

What is the purpose of this table?

Why does its primary key contain **two columns**?

Task A4: Generate a simple **ER diagram** showing the seven tables and their relationships.

Use the diagram to check the primary and foreign keys you identified in A1 and A2.

- In Supabase: From left-hand floating menu select Database. Cardinality is not shown. Relationships are as lines linking the attributes involved in the relationship. The following symbols appear as annotations:

 Primary key  Identity  Unique  Nullable  Non-Nullable

- In PGAdmin: Right-click on the schema name and choose ERD for Schema. You will have to zoom in and move the entities to get a clear diagram.

Task A5: Identify at least three constraints that protect data integrity.

Task A6: Can the same staff member have two appointments beginning at exactly the same date and time? Explain how the database enforces this.

Task A7: **(Optional)** Which columns allow NULL values?

Task A8: **(Optional)** What appointment status values are permitted?

6. Part B — SQL review

You studied SQL in Databases 1. The purpose of this section is to refresh those skills using the CommunityCare database. If you need a reminder of SQL syntax, you may refer to the PostgreSQL documentation

(<https://www.postgresql.org/docs/current/tutorial.html>) or this PostgreSQL Cheatsheet

(<https://neon.com/postgresqltutorial/PostgreSQL-Cheat-Sheet.pdf>).

Before starting, make sure your working schema is set: `SET search_path TO communitycare;`

Write and execute a query for each of the following Tasks.

Task B1: Simple retrieval

- List all patients, ordered by surname and then first name.

Hint: You only need one table. Use ORDER BY with two columns.

Task B2: Joining tables

- Show each appointment together with:
 - the patient's name;
 - the service name;
 - the appointment date;
 - the appointment time;
 - the appointment status.

Hint: This requires several tables. First work out the route between the tables using your ER diagram.

Start with appointment. Which foreign keys allow you to reach patient and service?

You will need more than one JOIN.

Think about ordering the output.

Task B3: A larger join

- Show all **cancelled appointments** together with:
 - the patient's name;
 - the staff member's name;
 - the service;
 - the appointment date.

Hint: This is similar to B2. It requires an additional table. It requires different columns in the output.

Use a WHERE condition to restrict the result to cancelled appointments.

Task B4: Grouping and counting

- Show the number of appointments for each service.
- Display service name and number of appointments
- Order the result from the service with the most appointments to the service with the fewest.

Hint: You will need:

- JOIN
- COUNT()
- GROUP BY
- ORDER BY

Task B5: Filtering groups

- Show only services that have **more than five appointments**.

Hint: Start with your solution to B4.

WHERE filters individual rows. Think about what clause filters the result **after the rows have been grouped**? Think about what the groups needs to *have*...

Task B6: Finding missing relationships

- Find patients who have **never had an appointment**.
- Display their patient ID, first name and surname.

Hint: An INNER JOIN only returns matching rows, so it is probably not what you need here.

Consider:

- LEFT JOIN
- What value will the appointment side of the join contain when a patient has no matching appointment?

Task B7: Finding missing payment records

- Show appointments for which **no payment record exists**.
- Display:
 - appointment ID;
 - patient name;
 - appointment date;
 - service name.

Hint: This is similar to B6.

Start with appointment and determine which tables need to be joined to it. Make sure appointments are retained even when there is **no matching row in payment**.

7. Part C — Problem that is harder than it first appears

Using your existing SQL knowledge, try to write the SQL needed to achieve the following:

Task C1. For each patient, show each appointment and the date of that patient's **previous appointment**.

Hints:

- Start by writing a query that displays each appointment together with the patient's name.
- For each appointment, you need to look again at the appointment table to find **other appointments for the same patient**.
- You are only interested in appointments that occurred **before the current appointment**.
- Of all the earlier appointment dates, which aggregate function could you use to find the **most recent** one?
- Think about whether a **subquery** could be used to calculate this value for each appointment.
- The first appointment for a patient will not have a previous appointment. What would you expect the query to return in that case?

You should be able to attempt this using SQL techniques from Databases 1.

Don't worry if your solution feels complicated or you cannot find a satisfactory solution.

We will later learn a cleaner, more concise way of solving this problem.

8. Part D — Transaction teaser

Consider two receptionists trying to book the same final appointment slot at the same time.

Receptionist A checks that 10:00 is available. Before A completes the booking, Receptionist B checks the same slot and also sees it as available. Both then attempt to book it.

Task D1. What could go wrong?

Task D2. Would simply putting each booking inside BEGIN and COMMIT necessarily solve the problem?

Task D3. What should the database guarantee when several users are accessing the same data concurrently?

Do not worry if you cannot yet give a complete technical solution. We will return to this scenario when studying transactions, ACID, isolation and locking.

9. Part E — A new requirement

CommunityCare wants patients to complete a pre-appointment questionnaire. The questions differ substantially between services.

A physiotherapy questionnaire might record:

- pain level;
- several pain areas;
- duration;
- previous treatment.

A dietitian questionnaire might record:

- allergies;
- dietary requirements;
- goals;
- a food diary.

Consider the following tasks. Make a brief note of your ideas. **Do not implement anything yet.**

Task E1. How might you store this information in the current relational database?

Task E2. What difficulties might arise if every service has a different set of questions?

We will return to this requirement later in the module.

10. Before you finish

Make sure you can:

- connect to CommunityCare after restarting your environment;
- see the seven supplied tables;
- run SQL queries against the data (remember to set your search path);
- explain the main relationships without relying on the schema script;
- save your SQL work for use in later labs.

Docker shutdown

If using Docker, either stop the containers:

```
docker compose stop
```

or remove the containers while retaining the database volume:

```
docker compose down
```

Your PostgreSQL data is retained when using either of these commands.